

Laboratory work 2 Normalization and DDL

Normalization

Task 1 Normal form violations in Booking receipt

Booking_receipt(booking_id, passenger_full_name, passenger_passport_number, flight_number, departure_airport_name, departure_city, arrival_airport_name, arrival_city, airline_name, seat_numbers, ticket_price)

The table violates 1NF because seat_numbers is not atomic. A value such as 12A, 12B stores a repeating group in one column, rather than one value per row and column.

Because the table is not in 1NF, it cannot be in 2NF or 3NF. Once seat_numbers is split into atomic rows, two further issues appear: partial dependencies on booking_id and passenger_passport_number, and transitive dependencies through flight_number, airport, and airline data.

Task 2 Step by step normalization

Step 1 Bring the relation to 1NF.

Replace every comma-separated seat_numbers list with separate rows. For example, a receipt containing 12A, 12B becomes two rows, each with one seat_number.

Booking_receipt_1NF(booking_id, passenger_passport_number, passenger_full_name, flight_number, departure_airport_name, departure_city, arrival_airport_name, arrival_city, airline_name, seat_number, ticket_price)

Primary key: (booking_id, passenger_passport_number). There are no foreign keys at this stage because the data remains in one relation.

Step 2 Bring the 1NF relation to 2NF.

The composite 1NF key has partial dependencies: booking_id determines the reservation and flight data, while passenger_passport_number determines passenger_full_name. Move those facts into their own tables and keep the seat assignment in the junction table.

Booking_2NF(booking_id, flight_number, departure_airport_name, departure_city, arrival_airport_name, arrival_city, airline_name, ticket_price)

Passenger_2NF(passenger_passport_number, passenger_full_name)

Booking_passenger_2NF(booking_id [FK -> Booking_2NF.booking_id], passenger_passport_number [FK -> Passenger_2NF.passenger_passport_number], seat_number)

Step 3 Bring the 2NF result to 3NF.

Booking_2NF still contains transitive dependencies. flight_number determines the airline and the departure/arrival airports; an airport determines its name and city; an airline determines its name. Decompose those entities as follows.

Airport_3NF(airport_id, airport_name, city)

Airline_3NF(airline_id, airline_name)

Flight_3NF(flight_number, departure_airport_id [FK -> Airport_3NF.airport_id], arrival_airport_id [FK -> Airport_3NF.airport_id], airline_id [FK -> Airline_3NF.airline_id])

Booking_3NF(booking_id, flight_number [FK -> Flight_3NF.flight_number], ticket_price)

Passenger_3NF(passenger_passport_number, passenger_full_name)

Booking_passenger_3NF(booking_id [FK -> Booking_3NF.booking_id], passenger_passport_number [FK -> Passenger_3NF.passenger_passport_number], seat_number)

Task 3 Functional dependencies

- booking_id -> flight_number, ticket_price
- passenger_passport_number -> passenger_full_name
- (booking_id, passenger_passport_number) -> seat_number
- flight_number -> departure_airport_id, arrival_airport_id, airline_id

- airport_id -> airport_name, city
- airline_id -> airline_name

These dependencies justify the decomposition: the first two are partial dependencies in the composite-key 1NF relation, and the last three remove the transitive dependencies from the booking data.

Task 4 Check whether Flights is in 3NF

Current primary key: flight_id.

- flight_id -> sch_departure_time, sch_arrival_time, departing_airport_id, arriving_airport_id, departing_gate, arriving_gate, airline_id, act_departure_time, act_arrival_time, created_at, updated_at

Under the stated design, Flights is in 3NF. Every non-key attribute depends on flight_id, and no non-key attribute in Flights determines another non-key attribute. departing_airport_id and arriving_airport_id are foreign keys to Airport, and airline_id is a foreign key to Airline; the descriptive airport and airline attributes are not stored in Flights. Therefore there is no transitive dependency inside Flights. Foreign keys themselves do not violate 3NF.

Task 5 PostgreSQL implementation of the 3NF Booking receipt schema

The SQL script creates the six br_ tables shown above and inserts sample data. It contains five bookings, five flights, and six passenger-seat rows; booking 1001 demonstrates a booking with two passengers and two atomic seat values.

Table	Primary key	Foreign keys
br_airport	airport_id	None
br_airline	airline_id	None
br_flight	flight_number	departure_airport_id, arrival_airport_id, airline_id
br_booking	booking_id	flight_number
br_passenger	passenger_passport_number	None
br_booking_passenger	booking_id, passenger_passport_number	booking_id, passenger_passport_number

DDL

The following implementation uses the exact table and column terminology in the assignment. Each requested attribute is NOT NULL, primary keys are defined for every table, and all listed foreign-key relationships are added after the requested renames and type changes.

Requirement	Implemented in the SQL script
Create the listed tables	airline_info, airport, baggage_check, baggage, boarding_pass, booking_flight, booking, flights, passengers, security_check
Primary keys and NOT NULL	Defined inline in every CREATE TABLE statement
Rename airline_info	ALTER TABLE airline_info RENAME TO airline
Rename price	ALTER TABLE booking RENAME COLUMN price TO ticket_price
Change departing_gate	ALTER TABLE flights ALTER COLUMN departing_gate TYPE TEXT
Drop info	ALTER TABLE airline DROP COLUMN info
Required relationships	Eleven foreign keys, matching every listed relationship

The Booking table includes flight_id because it is listed in the supplied attributes. The assignment explicitly requests the Flights relationship through Booking_flight, so the foreign-key section implements that requested relationship without adding an extra, unstated Booking.flight_id -> Flights.flight_id constraint.

```

-- Laboratory work 2: Normalization and DDL
-- PostgreSQL syntax

BEGIN;

-- =====
-- NORMALIZATION, TASK 5: 3NF schema for Booking_receipt
-- The br_ prefix keeps this schema independent from the DDL task.
-- =====

CREATE TABLE br_airport (
    airport_id INTEGER PRIMARY KEY,
    airport_name VARCHAR(100) NOT NULL,
    city VARCHAR(50) NOT NULL
);

CREATE TABLE br_airline (
    airline_id INTEGER PRIMARY KEY,
    airline_name VARCHAR(50) NOT NULL
);

CREATE TABLE br_flight (
    flight_number VARCHAR(10) PRIMARY KEY,
    departure_airport_id INTEGER NOT NULL,
    arrival_airport_id INTEGER NOT NULL,
    airline_id INTEGER NOT NULL,
    CONSTRAINT fk_br_flight_departure_airport
        FOREIGN KEY (departure_airport_id) REFERENCES br_airport (airport_id),
    CONSTRAINT fk_br_flight_arrival_airport
        FOREIGN KEY (arrival_airport_id) REFERENCES br_airport (airport_id),
    CONSTRAINT fk_br_flight_airline
        FOREIGN KEY (airline_id) REFERENCES br_airline (airline_id)
);

CREATE TABLE br_booking (
    booking_id INTEGER PRIMARY KEY,
    flight_number VARCHAR(10) NOT NULL,
    ticket_price DECIMAL(10, 2) NOT NULL,
    CONSTRAINT fk_br_booking_flight
        FOREIGN KEY (flight_number) REFERENCES br_flight (flight_number)
);

CREATE TABLE br_passenger (
    passenger_passport_number VARCHAR(20) PRIMARY KEY,
    passenger_full_name VARCHAR(100) NOT NULL
);

CREATE TABLE br_booking_passenger (
    booking_id INTEGER NOT NULL,
    passenger_passport_number VARCHAR(20) NOT NULL,
    seat_number VARCHAR(5) NOT NULL,
    CONSTRAINT pk_br_booking_passenger
        PRIMARY KEY (booking_id, passenger_passport_number),
    CONSTRAINT fk_br_booking_passenger_booking
        FOREIGN KEY (booking_id) REFERENCES br_booking (booking_id),
    CONSTRAINT fk_br_booking_passenger_passenger
        FOREIGN KEY (passenger_passport_number)
            REFERENCES br_passenger (passenger_passport_number)
);

-- Sample data: five bookings; booking 1001 has two passengers/seats.
INSERT INTO br_airport (airport_id, airport_name, city) VALUES
    (1, 'Almaty International Airport', 'Almaty'),
    (2, 'Nursultan Nazarbayev International Airport', 'Astana'),
    (3, 'Shymkent International Airport', 'Shymkent'),
    (4, 'Tashkent International Airport', 'Tashkent'),
    (5, 'Istanbul Airport', 'Istanbul');

INSERT INTO br_airline (airline_id, airline_name) VALUES
    (1, 'Air Astana'),
    (2, 'Uzbekistan Airways'),
    (3, 'Turkish Airlines');

INSERT INTO br_flight
    (flight_number, departure_airport_id, arrival_airport_id, airline_id)

```

```

VALUES
    ('KC101', 1, 2, 1),
    ('KC201', 2, 3, 1),
    ('HY762', 3, 4, 2),
    ('TK351', 1, 5, 3),
    ('KC855', 4, 1, 1);

INSERT INTO br_booking (booking_id, flight_number, ticket_price) VALUES
    (1001, 'KC101', 450.00),
    (1002, 'KC201', 120.00),
    (1003, 'HY762', 95.00),
    (1004, 'TK351', 310.00),
    (1005, 'KC855', 135.00);

INSERT INTO br_passenger
    (passenger_passport_number, passenger_full_name)
VALUES
    ('N1234567', 'Aruzhan Bekova'),
    ('N2345678', 'Dias Sarsembayev'),
    ('N3456789', 'Madina Tulegenova'),
    ('N4567890', 'Timur Nurgaliyev'),
    ('N5678901', 'Amina Kadyrova'),
    ('N6789012', 'Sanzhar Zhaksylykov');

INSERT INTO br_booking_passenger
    (booking_id, passenger_passport_number, seat_number)
VALUES
    (1001, 'N1234567', '12A'),
    (1001, 'N2345678', '12B'),
    (1002, 'N3456789', '08C'),
    (1003, 'N4567890', '14D'),
    (1004, 'N5678901', '22A'),
    (1005, 'N6789012', '10F');

-- =====
-- DDL, TASKS 1-9
-- Every listed attribute is NOT NULL and each table has a primary key.
-- =====

-- 1. Create the requested tables.
CREATE TABLE airline_info (
    airline_id INTEGER NOT NULL,
    airline_code VARCHAR(30) NOT NULL,
    airline_name VARCHAR(50) NOT NULL,
    airline_country VARCHAR(50) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    info VARCHAR(50) NOT NULL,
    CONSTRAINT pk_airline_info PRIMARY KEY (airline_id)
);

CREATE TABLE airport (
    airport_id INTEGER NOT NULL,
    airport_name VARCHAR(50) NOT NULL,
    country VARCHAR(50) NOT NULL,
    state VARCHAR(50) NOT NULL,
    city VARCHAR(50) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    CONSTRAINT pk_airport PRIMARY KEY (airport_id)
);

CREATE TABLE baggage_check (
    baggage_check_id INTEGER NOT NULL,
    check_result VARCHAR(50) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    booking_id INTEGER NOT NULL,
    passenger_id INTEGER NOT NULL,
    CONSTRAINT pk_baggage_check PRIMARY KEY (baggage_check_id)
);

CREATE TABLE baggage (
    baggage_id INTEGER NOT NULL,
    weight_in_kg DECIMAL(4, 2) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    booking_id INTEGER NOT NULL,

```

```

        CONSTRAINT pk_baggage PRIMARY KEY (baggage_id)
    );

CREATE TABLE boarding_pass (
    boarding_pass_id INTEGER NOT NULL,
    booking_id INTEGER NOT NULL,
    seat VARCHAR(50) NOT NULL,
    boarding_time TIMESTAMP NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    CONSTRAINT pk_boarding_pass PRIMARY KEY (boarding_pass_id)
);

CREATE TABLE booking_flight (
    booking_flight_id INTEGER NOT NULL,
    booking_id INTEGER NOT NULL,
    flight_id INTEGER NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    CONSTRAINT pk_booking_flight PRIMARY KEY (booking_flight_id)
);

CREATE TABLE booking (
    booking_id INTEGER NOT NULL,
    flight_id INTEGER NOT NULL,
    passenger_id INTEGER NOT NULL,
    booking_platform VARCHAR(50) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    status VARCHAR(50) NOT NULL,
    price DECIMAL(7, 2) NOT NULL,
    CONSTRAINT pk_booking PRIMARY KEY (booking_id)
);

CREATE TABLE flights (
    flight_id INTEGER NOT NULL,
    sch_departure_time TIMESTAMP NOT NULL,
    sch_arrival_time TIMESTAMP NOT NULL,
    departing_airport_id INTEGER NOT NULL,
    arriving_airport_id INTEGER NOT NULL,
    departing_gate VARCHAR(50) NOT NULL,
    arriving_gate VARCHAR(50) NOT NULL,
    airline_id INTEGER NOT NULL,
    act_departure_time TIMESTAMP NOT NULL,
    act_arrival_time TIMESTAMP NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    CONSTRAINT pk_flights PRIMARY KEY (flight_id)
);

CREATE TABLE passengers (
    passenger_id INTEGER NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    date_of_birth DATE NOT NULL,
    gender VARCHAR(50) NOT NULL,
    country_of_citizenship VARCHAR(50) NOT NULL,
    country_of_residence VARCHAR(50) NOT NULL,
    passport_number VARCHAR(20) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    CONSTRAINT pk_passengers PRIMARY KEY (passenger_id)
);

CREATE TABLE security_check (
    security_check_id INTEGER NOT NULL,
    check_result VARCHAR(20) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    passenger_id INTEGER NOT NULL,
    CONSTRAINT pk_security_check PRIMARY KEY (security_check_id)
);

-- 4. Rename airline_info to airline.
ALTER TABLE airline_info RENAME TO airline;

-- 5. Rename booking.price to booking.ticket_price.
ALTER TABLE booking RENAME COLUMN price TO ticket_price;

```

```

-- 6. Change Flights.departing_gate from varchar(50) to text.
ALTER TABLE flights
    ALTER COLUMN departing_gate TYPE TEXT;

-- 7. Drop airline.info.
ALTER TABLE airline
    DROP COLUMN info;

-- 8-9. Add every relationship requested in the assignment.
ALTER TABLE security_check
    ADD CONSTRAINT fk_security_check_passenger
    FOREIGN KEY (passenger_id) REFERENCES passengers (passenger_id);

ALTER TABLE booking
    ADD CONSTRAINT fk_booking_passenger
    FOREIGN KEY (passenger_id) REFERENCES passengers (passenger_id);

ALTER TABLE baggage_check
    ADD CONSTRAINT fk_baggage_check_passenger
    FOREIGN KEY (passenger_id) REFERENCES passengers (passenger_id);

ALTER TABLE baggage_check
    ADD CONSTRAINT fk_baggage_check_booking
    FOREIGN KEY (booking_id) REFERENCES booking (booking_id);

ALTER TABLE baggage
    ADD CONSTRAINT fk_baggage_booking
    FOREIGN KEY (booking_id) REFERENCES booking (booking_id);

ALTER TABLE boarding_pass
    ADD CONSTRAINT fk_boarding_pass_booking
    FOREIGN KEY (booking_id) REFERENCES booking (booking_id);

ALTER TABLE booking_flight
    ADD CONSTRAINT fk_booking_flight_booking
    FOREIGN KEY (booking_id) REFERENCES booking (booking_id);

ALTER TABLE booking_flight
    ADD CONSTRAINT fk_booking_flight_flight
    FOREIGN KEY (flight_id) REFERENCES flights (flight_id);

ALTER TABLE flights
    ADD CONSTRAINT fk_flights_departing_airport
    FOREIGN KEY (departing_airport_id) REFERENCES airport (airport_id);

ALTER TABLE flights
    ADD CONSTRAINT fk_flights_arriving_airport
    FOREIGN KEY (arriving_airport_id) REFERENCES airport (airport_id);

ALTER TABLE flights
    ADD CONSTRAINT fk_flights_airline
    FOREIGN KEY (airline_id) REFERENCES airline (airline_id);

COMMIT;

```

Laboratory work 2 Normalization and DDL

Normalization

Task 1 Normal form violations in Booking receipt

Booking_receipt(booking_id, passenger_full_name, passenger_passport_number, flight_number, departure_airport_name, departure_city, arrival_airport_name, arrival_city, airline_name, seat_numbers, ticket_price)

The table violates 1NF because seat_numbers is not atomic. A value such as 12A, 12B stores a repeating group in one column, rather than one value per row and column.

Because the table is not in 1NF, it cannot be in 2NF or 3NF. Once seat_numbers is split into atomic rows, two further issues appear: partial dependencies on booking_id and passenger_passport_number, and transitive dependencies through flight_number, airport, and airline data.

Task 2 Step by step normalization

Step 1 Bring the relation to 1NF.

Replace every comma-separated seat_numbers list with separate rows. For example, a receipt containing 12A, 12B becomes two rows, each with one seat_number.

Booking_receipt_1NF(booking_id, passenger_passport_number, passenger_full_name, flight_number, departure_airport_name, departure_city, arrival_airport_name, arrival_city, airline_name, seat_number, ticket_price)

Primary key: (booking_id, passenger_passport_number). There are no foreign keys at this stage because the data remains in one relation.

Step 2 Bring the 1NF relation to 2NF.

The composite 1NF key has partial dependencies: booking_id determines the reservation and flight data, while passenger_passport_number determines passenger_full_name. Move those facts into their own tables and keep the seat assignment in the junction table.

Booking_2NF(booking_id, flight_number, departure_airport_name, departure_city, arrival_airport_name, arrival_city, airline_name, ticket_price)

Passenger_2NF(passenger_passport_number, passenger_full_name)

Booking_passenger_2NF(booking_id [FK -> Booking_2NF.booking_id], passenger_passport_number [FK -> Passenger_2NF.passenger_passport_number], seat_number)

Step 3 Bring the 2NF result to 3NF.

Booking_2NF still contains transitive dependencies. flight_number determines the airline and the departure/arrival airports; an airport determines its name and city; an airline determines its name. Decompose those entities as follows.

Airport_3NF(airport_id, airport_name, city)

Airline_3NF(airline_id, airline_name)

Flight_3NF(flight_number, departure_airport_id [FK -> Airport_3NF.airport_id], arrival_airport_id [FK -> Airport_3NF.airport_id], airline_id [FK -> Airline_3NF.airline_id])

Booking_3NF(booking_id, flight_number [FK -> Flight_3NF.flight_number], ticket_price)

Passenger_3NF(passenger_passport_number, passenger_full_name)

Booking_passenger_3NF(booking_id [FK -> Booking_3NF.booking_id], passenger_passport_number [FK -> Passenger_3NF.passenger_passport_number], seat_number)

Task 3 Functional dependencies

- booking_id -> flight_number, ticket_price
- passenger_passport_number -> passenger_full_name
- (booking_id, passenger_passport_number) -> seat_number
- flight_number -> departure_airport_id, arrival_airport_id, airline_id

- airport_id -> airport_name, city
- airline_id -> airline_name

These dependencies justify the decomposition: the first two are partial dependencies in the composite-key 1NF relation, and the last three remove the transitive dependencies from the booking data.

Task 4 Check whether Flights is in 3NF

Current primary key: flight_id.

- flight_id -> sch_departure_time, sch_arrival_time, departing_airport_id, arriving_airport_id, departing_gate, arriving_gate, airline_id, act_departure_time, act_arrival_time, created_at, updated_at

Under the stated design, Flights is in 3NF. Every non-key attribute depends on flight_id, and no non-key attribute in Flights determines another non-key attribute. departing_airport_id and arriving_airport_id are foreign keys to Airport, and airline_id is a foreign key to Airline; the descriptive airport and airline attributes are not stored in Flights. Therefore there is no transitive dependency inside Flights. Foreign keys themselves do not violate 3NF.

Task 5 PostgreSQL implementation of the 3NF Booking receipt schema

The SQL script creates the six br_ tables shown above and inserts sample data. It contains five bookings, five flights, and six passenger-seat rows; booking 1001 demonstrates a booking with two passengers and two atomic seat values.

Table	Primary key	Foreign keys
br_airport	airport_id	None
br_airline	airline_id	None
br_flight	flight_number	departure_airport_id, arrival_airport_id, airline_id
br_booking	booking_id	flight_number
br_passenger	passenger_passport_number	None
br_booking_passenger	booking_id, passenger_passport_number	booking_id, passenger_passport_number

DDL

The following implementation uses the exact table and column terminology in the assignment. Each requested attribute is NOT NULL, primary keys are defined for every table, and all listed foreign-key relationships are added after the requested renames and type changes.

Requirement	Implemented in the SQL script
Create the listed tables	airline_info, airport, baggage_check, baggage, boarding_pass, booking_flight, booking, flights, passengers, security_check
Primary keys and NOT NULL	Defined inline in every CREATE TABLE statement
Rename airline_info	ALTER TABLE airline_info RENAME TO airline
Rename price	ALTER TABLE booking RENAME COLUMN price TO ticket_price
Change departing_gate	ALTER TABLE flights ALTER COLUMN departing_gate TYPE TEXT
Drop info	ALTER TABLE airline DROP COLUMN info
Required relationships	Eleven foreign keys, matching every listed relationship

The Booking table includes flight_id because it is listed in the supplied attributes. The assignment explicitly requests the Flights relationship through Booking_flight, so the foreign-key section implements that requested relationship without adding an extra, unstated Booking.flight_id -> Flights.flight_id constraint.

```

-- Laboratory work 2: Normalization and DDL
-- PostgreSQL syntax

BEGIN;

-- =====
-- NORMALIZATION, TASK 5: 3NF schema for Booking_receipt
-- The br_ prefix keeps this schema independent from the DDL task.
-- =====

CREATE TABLE br_airport (
    airport_id INTEGER PRIMARY KEY,
    airport_name VARCHAR(100) NOT NULL,
    city VARCHAR(50) NOT NULL
);

CREATE TABLE br_airline (
    airline_id INTEGER PRIMARY KEY,
    airline_name VARCHAR(50) NOT NULL
);

CREATE TABLE br_flight (
    flight_number VARCHAR(10) PRIMARY KEY,
    departure_airport_id INTEGER NOT NULL,
    arrival_airport_id INTEGER NOT NULL,
    airline_id INTEGER NOT NULL,
    CONSTRAINT fk_br_flight_departure_airport
        FOREIGN KEY (departure_airport_id) REFERENCES br_airport (airport_id),
    CONSTRAINT fk_br_flight_arrival_airport
        FOREIGN KEY (arrival_airport_id) REFERENCES br_airport (airport_id),
    CONSTRAINT fk_br_flight_airline
        FOREIGN KEY (airline_id) REFERENCES br_airline (airline_id)
);

CREATE TABLE br_booking (
    booking_id INTEGER PRIMARY KEY,
    flight_number VARCHAR(10) NOT NULL,
    ticket_price DECIMAL(10, 2) NOT NULL,
    CONSTRAINT fk_br_booking_flight
        FOREIGN KEY (flight_number) REFERENCES br_flight (flight_number)
);

CREATE TABLE br_passenger (
    passenger_passport_number VARCHAR(20) PRIMARY KEY,
    passenger_full_name VARCHAR(100) NOT NULL
);

CREATE TABLE br_booking_passenger (
    booking_id INTEGER NOT NULL,
    passenger_passport_number VARCHAR(20) NOT NULL,
    seat_number VARCHAR(5) NOT NULL,
    CONSTRAINT pk_br_booking_passenger
        PRIMARY KEY (booking_id, passenger_passport_number),
    CONSTRAINT fk_br_booking_passenger_booking
        FOREIGN KEY (booking_id) REFERENCES br_booking (booking_id),
    CONSTRAINT fk_br_booking_passenger_passenger
        FOREIGN KEY (passenger_passport_number)
            REFERENCES br_passenger (passenger_passport_number)
);

-- Sample data: five bookings; booking 1001 has two passengers/seats.
INSERT INTO br_airport (airport_id, airport_name, city) VALUES
    (1, 'Almaty International Airport', 'Almaty'),
    (2, 'Nursultan Nazarbayev International Airport', 'Astana'),
    (3, 'Shymkent International Airport', 'Shymkent'),
    (4, 'Tashkent International Airport', 'Tashkent'),
    (5, 'Istanbul Airport', 'Istanbul');

INSERT INTO br_airline (airline_id, airline_name) VALUES
    (1, 'Air Astana'),
    (2, 'Uzbekistan Airways'),
    (3, 'Turkish Airlines');

INSERT INTO br_flight
    (flight_number, departure_airport_id, arrival_airport_id, airline_id)

```

```

VALUES
    ('KC101', 1, 2, 1),
    ('KC201', 2, 3, 1),
    ('HY762', 3, 4, 2),
    ('TK351', 1, 5, 3),
    ('KC855', 4, 1, 1);

INSERT INTO br_booking (booking_id, flight_number, ticket_price) VALUES
    (1001, 'KC101', 450.00),
    (1002, 'KC201', 120.00),
    (1003, 'HY762', 95.00),
    (1004, 'TK351', 310.00),
    (1005, 'KC855', 135.00);

INSERT INTO br_passenger
    (passenger_passport_number, passenger_full_name)
VALUES
    ('N1234567', 'Aruzhan Bekova'),
    ('N2345678', 'Dias Sarsembayev'),
    ('N3456789', 'Madina Tulegenova'),
    ('N4567890', 'Timur Nurgaliyev'),
    ('N5678901', 'Amina Kadyrova'),
    ('N6789012', 'Sanzhar Zhaksylykov');

INSERT INTO br_booking_passenger
    (booking_id, passenger_passport_number, seat_number)
VALUES
    (1001, 'N1234567', '12A'),
    (1001, 'N2345678', '12B'),
    (1002, 'N3456789', '08C'),
    (1003, 'N4567890', '14D'),
    (1004, 'N5678901', '22A'),
    (1005, 'N6789012', '10F');

-- =====
-- DDL, TASKS 1-9
-- Every listed attribute is NOT NULL and each table has a primary key.
-- =====

-- 1. Create the requested tables.
CREATE TABLE airline_info (
    airline_id INTEGER NOT NULL,
    airline_code VARCHAR(30) NOT NULL,
    airline_name VARCHAR(50) NOT NULL,
    airline_country VARCHAR(50) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    info VARCHAR(50) NOT NULL,
    CONSTRAINT pk_airline_info PRIMARY KEY (airline_id)
);

CREATE TABLE airport (
    airport_id INTEGER NOT NULL,
    airport_name VARCHAR(50) NOT NULL,
    country VARCHAR(50) NOT NULL,
    state VARCHAR(50) NOT NULL,
    city VARCHAR(50) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    CONSTRAINT pk_airport PRIMARY KEY (airport_id)
);

CREATE TABLE baggage_check (
    baggage_check_id INTEGER NOT NULL,
    check_result VARCHAR(50) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    booking_id INTEGER NOT NULL,
    passenger_id INTEGER NOT NULL,
    CONSTRAINT pk_baggage_check PRIMARY KEY (baggage_check_id)
);

CREATE TABLE baggage (
    baggage_id INTEGER NOT NULL,
    weight_in_kg DECIMAL(4, 2) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    booking_id INTEGER NOT NULL,

```

```

        CONSTRAINT pk_baggage PRIMARY KEY (baggage_id)
    );

CREATE TABLE boarding_pass (
    boarding_pass_id INTEGER NOT NULL,
    booking_id INTEGER NOT NULL,
    seat VARCHAR(50) NOT NULL,
    boarding_time TIMESTAMP NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    CONSTRAINT pk_boarding_pass PRIMARY KEY (boarding_pass_id)
);

CREATE TABLE booking_flight (
    booking_flight_id INTEGER NOT NULL,
    booking_id INTEGER NOT NULL,
    flight_id INTEGER NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    CONSTRAINT pk_booking_flight PRIMARY KEY (booking_flight_id)
);

CREATE TABLE booking (
    booking_id INTEGER NOT NULL,
    flight_id INTEGER NOT NULL,
    passenger_id INTEGER NOT NULL,
    booking_platform VARCHAR(50) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    status VARCHAR(50) NOT NULL,
    price DECIMAL(7, 2) NOT NULL,
    CONSTRAINT pk_booking PRIMARY KEY (booking_id)
);

CREATE TABLE flights (
    flight_id INTEGER NOT NULL,
    sch_departure_time TIMESTAMP NOT NULL,
    sch_arrival_time TIMESTAMP NOT NULL,
    departing_airport_id INTEGER NOT NULL,
    arriving_airport_id INTEGER NOT NULL,
    departing_gate VARCHAR(50) NOT NULL,
    arriving_gate VARCHAR(50) NOT NULL,
    airline_id INTEGER NOT NULL,
    act_departure_time TIMESTAMP NOT NULL,
    act_arrival_time TIMESTAMP NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    CONSTRAINT pk_flights PRIMARY KEY (flight_id)
);

CREATE TABLE passengers (
    passenger_id INTEGER NOT NULL,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    date_of_birth DATE NOT NULL,
    gender VARCHAR(50) NOT NULL,
    country_of_citizenship VARCHAR(50) NOT NULL,
    country_of_residence VARCHAR(50) NOT NULL,
    passport_number VARCHAR(20) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    CONSTRAINT pk_passengers PRIMARY KEY (passenger_id)
);

CREATE TABLE security_check (
    security_check_id INTEGER NOT NULL,
    check_result VARCHAR(20) NOT NULL,
    created_at TIMESTAMP NOT NULL,
    updated_at TIMESTAMP NOT NULL,
    passenger_id INTEGER NOT NULL,
    CONSTRAINT pk_security_check PRIMARY KEY (security_check_id)
);

-- 4. Rename airline_info to airline.
ALTER TABLE airline_info RENAME TO airline;

-- 5. Rename booking.price to booking.ticket_price.
ALTER TABLE booking RENAME COLUMN price TO ticket_price;

```

```

-- 6. Change Flights.departing_gate from varchar(50) to text.
ALTER TABLE flights
    ALTER COLUMN departing_gate TYPE TEXT;

-- 7. Drop airline.info.
ALTER TABLE airline
    DROP COLUMN info;

-- 8-9. Add every relationship requested in the assignment.
ALTER TABLE security_check
    ADD CONSTRAINT fk_security_check_passenger
    FOREIGN KEY (passenger_id) REFERENCES passengers (passenger_id);

ALTER TABLE booking
    ADD CONSTRAINT fk_booking_passenger
    FOREIGN KEY (passenger_id) REFERENCES passengers (passenger_id);

ALTER TABLE baggage_check
    ADD CONSTRAINT fk_baggage_check_passenger
    FOREIGN KEY (passenger_id) REFERENCES passengers (passenger_id);

ALTER TABLE baggage_check
    ADD CONSTRAINT fk_baggage_check_booking
    FOREIGN KEY (booking_id) REFERENCES booking (booking_id);

ALTER TABLE baggage
    ADD CONSTRAINT fk_baggage_booking
    FOREIGN KEY (booking_id) REFERENCES booking (booking_id);

ALTER TABLE boarding_pass
    ADD CONSTRAINT fk_boarding_pass_booking
    FOREIGN KEY (booking_id) REFERENCES booking (booking_id);

ALTER TABLE booking_flight
    ADD CONSTRAINT fk_booking_flight_booking
    FOREIGN KEY (booking_id) REFERENCES booking (booking_id);

ALTER TABLE booking_flight
    ADD CONSTRAINT fk_booking_flight_flight
    FOREIGN KEY (flight_id) REFERENCES flights (flight_id);

ALTER TABLE flights
    ADD CONSTRAINT fk_flights_departing_airport
    FOREIGN KEY (departing_airport_id) REFERENCES airport (airport_id);

ALTER TABLE flights
    ADD CONSTRAINT fk_flights_arriving_airport
    FOREIGN KEY (arriving_airport_id) REFERENCES airport (airport_id);

ALTER TABLE flights
    ADD CONSTRAINT fk_flights_airline
    FOREIGN KEY (airline_id) REFERENCES airline (airline_id);

COMMIT;

```